

Instruções para iniciar, testar e encerrar a aplicação

Objetivo

Iniciar os três serviços do Rota Fluminense com Docker Compose, testar a aplicação exclusivamente pela interface React e pelo Swagger e encerrar os contêineres sem apagar os dados do MySQL.

Não são usados curl, Invoke-WebRequest, scripts de teste ou acesso direto ao banco neste roteiro.

O fluxo canônico parte da raiz de rota-fluminense-front-end-avancado, onde o docker-compose.yml atende à estrutura de entrega. O Compose equivalente mantido no back-end continua disponível para compatibilidade operacional.

Pré-requisitos

- Docker Desktop ou Docker Engine com o plugin Compose em execução.
- Repositórios rota-fluminense-backend e rota-fluminense-front-end-avancado em diretórios irmãos.
- Terminal aberto na raiz de rota-fluminense-front-end-avancado.
- Arquivo ../rota-fluminense-backend/.env configurado. Ele não deve ser exibido nem versionado.

Se o .env do back-end ainda não existir, crie-o a partir do exemplo e revise os valores localmente:

```
if (-not (Test-Path ../rota-fluminense-backend/.env)) { Copy-Item ../rota-fluminense-backend/.env.example  
  ../rota-fluminense-backend/.env }
```

1. Iniciar a aplicação

O comando padrão abaixo não exige certificado adicional. Em ambientes com inspeção HTTPS por antivírus ou proxy corporativo, consulte a seção CA adicional para ambientes com inspeção HTTPS do README e acrescente o arquivo compose.custom-ca.example.yml; não desative a verificação TLS.

Valide a configuração antes de criar os contêineres:

```
docker compose --env-file ../rota-fluminense-backend/.env config --quiet
```

O comando termina sem saída quando a configuração é válida. Em seguida, construa as imagens e aguarde os serviços ficarem saudáveis:

```
docker compose --env-file ../rota-fluminense-backend/.env up --build --detach --wait  
docker compose --env-file ../rota-fluminense-backend/.env ps
```

Confirme em ps ou no Docker Desktop que os serviços frontend, backend e mysql estão em execução. O agrupamento Compose deve aparecer como rota-fluminense.

Abra somente estes endereços no navegador:

- interface: http://localhost:5173;
- Swagger: http://localhost:5000/openapi/.

1.1 Inicialização opcional com CA adicional

Use esta opção somente se o antivírus ou um proxy corporativo interceptar a conexão HTTPS do contêiner com o Open-Meteo. Não é necessário criar uma exceção no antivírus nem desativar a validação TLS.

Exporte a CA raiz utilizada pela inspeção HTTPS no formato PEM/Base-64 com a extensão .crt e mantenha o arquivo fora do Git. Depois, acrescente ao ../rota-fluminense-backend/.env o caminho absoluto do certificado

no computador hospedeiro:

```
OPEN_METEO_CA_HOST_PATH=C:/caminho/para/proxy-root-ca.crt
```

Valide a configuração combinada:

```
docker compose --env-file ../rota-fluminense-backend/.env `
-f docker-compose.yml `
-f ../rota-fluminense-backend/compose.custom-ca.example.yml `
config --quiet
```

Em seguida, construa e inicie a aplicação com o arquivo adicional:

```
docker compose --env-file ../rota-fluminense-backend/.env `
-f docker-compose.yml `
-f ../rota-fluminense-backend/compose.custom-ca.example.yml `
up --build --detach --wait

docker compose --env-file ../rota-fluminense-backend/.env `
-f docker-compose.yml `
-f ../rota-fluminense-backend/compose.custom-ca.example.yml `
ps
```

O certificado é montado no contêiner do backend somente para leitura e é adicionado às autoridades padrão já reconhecidas pela aplicação. Se não houver inspeção HTTPS, não defina `OPEN_METEO_CA_HOST_PATH` e utilize apenas o comando padrão da seção anterior.

2. Testes pela interface visual

2.1 Home, listagem e filtros

1. Acesse `http://localhost:5173`.
2. Confirme que a página apresenta os destinos sem mensagem de erro.
3. Use a busca textual e confirme que a lista é atualizada.
4. Selecione uma categoria e confira que apenas os locais correspondentes permanecem visíveis.
5. Limpe busca e filtro e confirme o retorno da listagem completa.

Resultado esperado: cards legíveis, imagens, cidade, categoria, nota e total de avaliações coerentes; nenhum fallback de dados simulados.

2.2 Detalhe e clima

1. Abra o detalhe do local **Arpoador**.
2. Confirme nome, descrição, localização e avaliações.
3. Confira o card climático, a condição atual, a previsão de três dias e a atribuição ao Open-Meteo.

Resultado esperado: o detalhe é carregado pela API e o clima aparece de forma independente. Se o provedor estiver indisponível, a interface deve mostrar uma mensagem clara sem impedir a leitura dos demais dados do local.

2.3 CRUD de avaliação pela interface

1. No detalhe do Arpoador, crie uma avaliação com autor `Teste Visual Front`, nota 5 e comentário `Avaliação criada pela interface visual..`
2. Confirme que ela aparece na lista e que total e média são atualizados.
3. Edite a mesma avaliação, alterando a nota para 4 e o comentário para `Avaliação editada pela interface visual..`
4. Confirme a atualização na tela.

5. Solicite a exclusão, confira a confirmação visual e exclua o registro.
6. Confirme que a avaliação desapareceu e que os agregados foram reconciliados.

Resultado esperado: criação, edição e exclusão produzem feedback visível e os botões ficam protegidos contra envios duplicados.

3. Testes pelo Swagger

Acesse <http://localhost:5000/openapi/>. Em cada operação, expanda a rota, clique em **Try it out**, preencha os parâmetros e use **Execute**. Não é necessária autenticação neste MVP.

3.1 Consultas GET

Execute nesta ordem:

1. GET /locais — resposta esperada 200, com locais e paginacao.
2. GET /locais/{slug} com slug = arpoador — resposta esperada 200.
3. GET /locais/{slug}/avaliacoes com slug = arpoador — resposta esperada 200, com o vetor avaliacoes.
4. GET /locais/{slug}/clima com slug = arpoador — resposta esperada 200, com clima atual e três dias de previsão. Uma indisponibilidade externa real pode produzir 503 documentado e não deve ser ocultada.

3.2 Inserção com POST

Expanda POST /locais/{slug}/avaliacoes, clique em **Try it out** e informe:

- slug: arpoador;
- corpo JSON:

```
{
  "autor": "Teste Visual Swagger",
  "nota": 5,
  "comentario": "Avaliação criada manualmente pelo Swagger."
}
```

Clique em **Execute**.

Resultado esperado: HTTP 201. A resposta contém id, local_id, autor, nota, comentario e criado_em. Anote o valor de id, pois ele identifica exatamente o registro que será editado e removido nos passos seguintes.

Volte ao detalhe do Arpoador no front-end e atualize a página. A avaliação Teste Visual Swagger deve aparecer, comprovando a integração entre Swagger, API, MySQL e interface.

3.3 Atualização com PATCH

Expanda PATCH /avaliacoes/{avaliacao_id}, use o id retornado pelo POST e envie:

```
{
  "nota": 4,
  "comentario": "Avaliação atualizada manualmente pelo Swagger."
}
```

Resultado esperado: HTTP 200, mantendo o mesmo id e retornando os campos atualizados. Atualize o detalhe no front-end e confirme a alteração visual.

3.4 Exclusão e limpeza do dado de teste

Expanda DELETE /avaliacoes/{avaliacao_id}, informe o mesmo id e execute.

Resultado esperado: HTTP 204, sem corpo. Atualize o front-end e confirme que Teste Visual Swagger não aparece mais.

Se o POST tiver sido executado, não encerre o roteiro antes de remover esse registro. Não use um identificador diferente daquele retornado pela criação.

4. Encerrar o Docker

Na raiz de `rota-fluminense-front-end-avancado`, encerre os contêineres e as redes do projeto:

```
docker compose --env-file ../rota-fluminense-backend/.env down
```

Esse comando preserva o volume `rota-fluminense_mysql_data` e, portanto, os dados persistidos no MySQL.

Confirme o encerramento:

```
docker compose --env-file ../rota-fluminense-backend/.env ps
```

O comando não deve listar contêineres ativos do projeto.

Não execute `docker compose down --volumes` no fluxo normal. A opção `--volumes` apaga o volume do banco e deve ser usada somente quando houver intenção explícita de reinicializar todos os dados.

Checklist final

- ☐ frontend, backend e mysql iniciaram corretamente.
- ☐ Home, busca, filtro, detalhe e clima foram conferidos visualmente.
- ☐ A avaliação criada pelo front-end foi editada e excluída.
- ☐ As quatro consultas GET foram executadas no Swagger.
- ☐ O POST retornou 201 e o registro apareceu no front-end.
- ☐ O PATCH retornou 200 e a alteração apareceu no front-end.
- ☐ O DELETE retornou 204 e removeu o registro de teste.
- ☐ `docker compose down` encerrou o projeto sem remover o volume do MySQL.